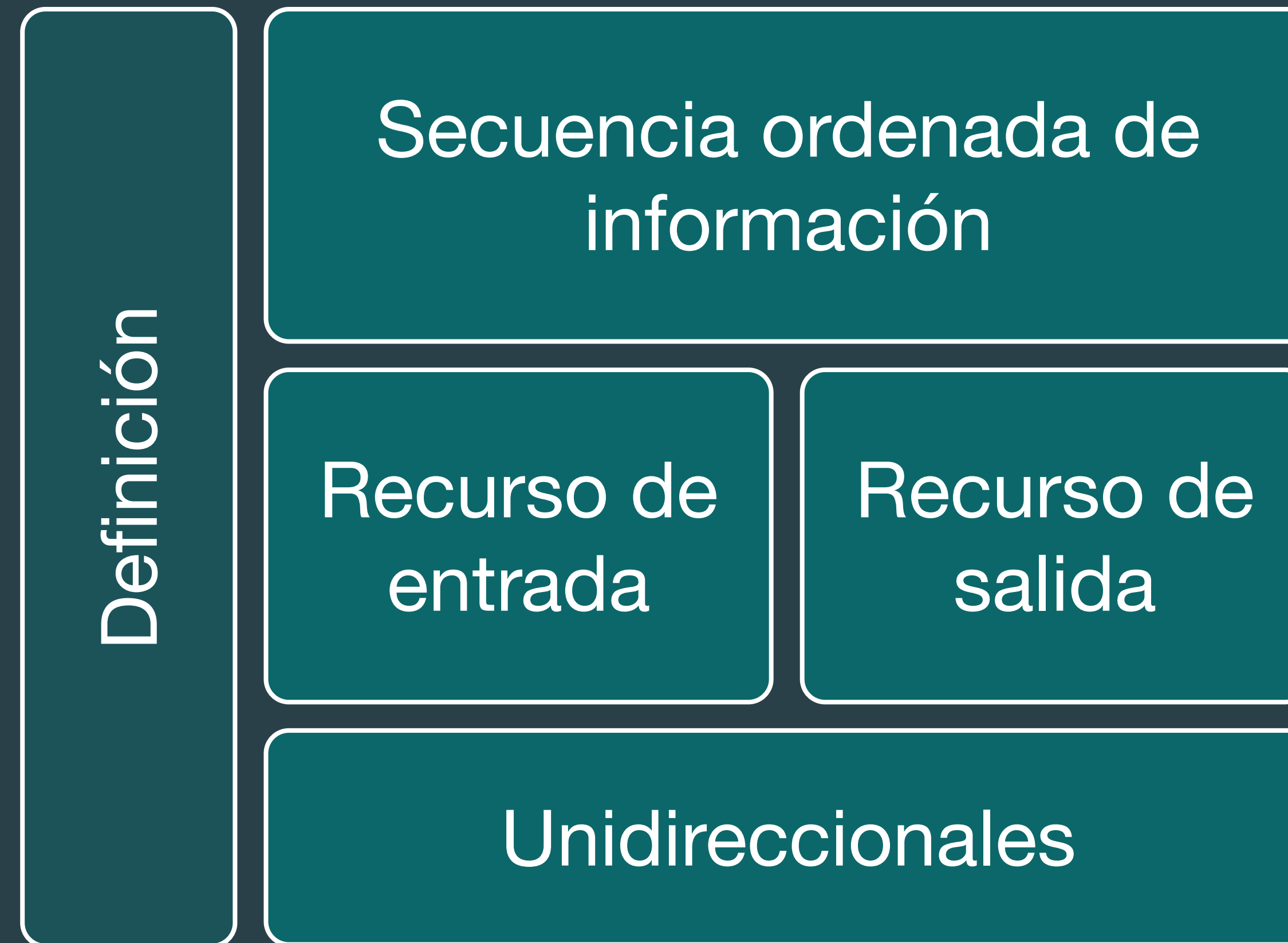


Tema 2

Flujos

Acceso a datos.

1. Definición de flujo/stream



Clases de flujos basados en ficheros

2. Clasificación de los Streams: Ficheros

Streams vistos en el tema 1

Utilidad

Bytes

Caracteres

Ficheros

FileInputStream,
FileOutputStream

FileReader,
FileWriter

¿Es randomAccessFile un Stream?

Clases de flujos basados en tuberías

2. Clasificación de los Streams: Tuberías

Tubería

Mecanismos que permite la comunicación entre dos hilos (threads) de un mismo programa.

Necesario para la comunicación: un mismo proceso y dos hilos diferentes

Utilidad

Bytes

Caracteres

Tuberías

PipedInputStream,
PipedOutputStream

PipedReader,
PipedWriter

Ejemplo: Hilo 1 escribe en PipedOutputStream

```
final PipedOutputStream salida = new PipedOutputStream();
final PipedInputStream entrada;

try {
    entrada = new PipedInputStream(salida);
} catch (IOException e) {
    e.printStackTrace();
    return;
}

Thread thread1 = new Thread(new Runnable() {
    @Override
    public void run() {
        try {
            salida.write("Hola por aquí!".getBytes());
        } catch (IOException e) {
        }
    }
});
```

1. Creamos un Stream de salida y otro de entrada conectados entre sí.
(cualquier dato escrito en el PipedOutputStream se puede leer inmediatamente desde el PipedInputStream).

2. Se crea un hilo que escribe en el PipedOutputStream un mensaje

Ejemplo: Hilo 2 lee de PipedInputStream

```
Thread thread2 = new Thread(new Runnable() {
    @Override
    public void run() {
        try {
            if(entrada != null) {
                int unByte = entrada.read();
                while (unByte != -1) {
                    System.out.print((char) unByte);
                    unByte = entrada.read();
                }
            }
        } catch (IOException e) {
        }
    }
});
thread1.start();
thread2.start();
```

1. Lee los datos de PipedInputStream (entrada/lectura)

2. Lo imprime en consola

Clases de flujos basados en arrays

3. Clasificación de los Streams: Arrays

Permiten leer y escribir datos en arrays de caracteres.

Utilidad

Bytes

Caracteres

Arrays

ByteArrayInputStream,
ByteArrayOutputStream

CharArrayReader,
CharArrayWriter

Ejemplo

```
public static void main(String[] args) {  
    char[] texto = "texto de ejemplo".toCharArray();  
    CharArrayReader charArrayReader = new CharArrayReader(texto);  
  
    int data = 0;  
    try {  
        data = charArrayReader.read();  
        while(data != -1) {  
            System.out.print((char)data);  
            data = charArrayReader.read();  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
  
    charArrayReader.close();  
}
```

1. Creamos un CharArrayReader utilizando el array de caracteres.
CharArrayReader permite leer datos desde un array de caracteres.

2. Leemos los datos y los mostramos por consola

Ejemplo

```
CharArrayWriter charArrayWriter = new CharArrayWriter();

try {
    charArrayWriter.write("texto de ejemplo 2");
} catch (IOException e) {
    e.printStackTrace();
}

char[] writer = charArrayWriter.toCharArray();

System.out.println(new String(writer));
charArrayWriter.close();
```

1. Creamos un CharArrayWriter y almacenamos un texto de ejemplo.

2. Los pasamos a un array de caracteres y lo mostramos.

Clases de análisis para flujos de datos Parte 1.

4. Clasificación de los Streams: Análisis

Permiten analizar ciertas partes de flujos de datos.

Utilidad

Bytes

Caracteres

Análisis

PushBackInputStream,
StreamTokenizer

PushBackReader,
LineNumberReader

4. Clasificación de los Streams: Análisis

Bytes

Caracteres

PushbackInputStream

PushbackReader

Análisis de datos previo: A veces es necesario leer algo de información para saber que se aproxima e interpretar la información actual.

Los bytes leídos luego son empujados de vuelta al flujo para poder seguir haciendo read(). (pushBack)

Ejemplo

```
try {
    PushbackReader pushbackReader = new PushbackReader(
        new FileReader("tema2.txt"));

    int data = pushbackReader.read();
    System.out.println((char)data);

    pushbackReader.unread(data);
    data = pushbackReader.read();

    System.out.println((char)data);
    pushbackReader.close();
} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}
```

1. Creamos un pushbackReader que contiene la información del archivo pasado.

2. Podemos leer la primera letra del fichero

3. Podemos devolver al stream de datos la letra leída.

4. Ambos println muestran la primera letra.

Clases de análisis para flujos de datos Parte 2.

4. Clasificación de los Streams: Análisis

Utilidad

Bytes

Análisis

StreamTokenizer

Permite analizar un determinado fichero por fragmentos.

Permite identificar números, comillas, espacios, etc.

Algunos campos que reconoce

TT_EOF

Fin del stream

TT_EOL

Fin de línea

TT_NUMBER

Token numérico

TT_WORD

Token es un conjunto de
letras

Ejemplo 1: Lectura desde String

```
StreamTokenizer streamTokenizer = new StreamTokenizer(  
    new StringReader("Este es el texto de ejemplo numero 12"));  
  
try {  
    while(streamTokenizer.nextToken() != streamTokenizer.TT_EOF) {  
        if(streamTokenizer.ttype == StreamTokenizer.TT_WORD) {  
            System.out.print("Letra: ");  
            System.out.println(streamTokenizer.sval);  
        }  
        else if(streamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {  
            System.out.print("Numero: ");  
            System.out.println(streamTokenizer.nval);  
        }  
    }  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Se crea un streamTokenizer a partir de la lectura de un String y se van distinguiendo sus partes (palabras y números).

Ejemplo 2: Lectura de fichero

```
StreamTokenizer streamTokenizer = new StreamTokenizer(  
    new FileInputStream("tema2_practica.txt"));
```

```
while(streamTokenizer.nextToken() != StreamTokenizer.TT_EOF) {  
}
```

```
if(streamTokenizer.ttype == StreamTokenizer.TT_WORD)
```

```
if(streamTokenizer.ttype == '>')  
if(streamTokenizer.ttype == ',')
```

```
if(streamTokenizer.ttype == StreamTokenizer.TT_NUMBER)
```

Creamos un streamTokenizer leyendo de un fichero

De esta forma podemos leer el fichero hasta el final

Podemos comprobar si estamos ante una palabra

Podemos comprobar si estamos ante un carácter concreto ('>' o ',')

Podemos comprobar si estamos ante un número

Ejemplo 2: Lectura de fichero

```
if(streamTokenizer.ttype == StreamTokenizer.TT_WORD) {  
    System.out.print(streamTokenizer.sval);  
}
```

Si estamos ante una palabra,
podemos mostrarla

```
if(streamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {  
    int number = (int) streamTokenizer.nval;  
}
```

Si estamos ante un número,
podemos obtenerlo

Clases de análisis para flujos de datos Parte 3.

4. Clasificación de los Streams: Análisis

Utilidad

Caracteres

Análisis

LineNumberReader

Permite contar el número de líneas leídas de caracteres. Tenemos dos métodos importantes:

`getLineNumber()`

`setLineNumber()`

4. Clasificación de los Streams: Análisis

```
try {
    LineNumberReader lineNumberReader =
        new LineNumberReader(new FileReader("tema2.txt"));
    String line = lineNumberReader.readLine();
    while(line != null) {
        System.out.println("Contenido de la linea numero:"
                           + lineNumberReader.getLineNumber());
        System.out.println(line);
        line = lineNumberReader.readLine();
    }
    lineNumberReader.close();
} catch (IOException e) {
    e.printStackTrace();
}
```

Se lee un archivo línea a línea mostrando que línea se está leyendo

Clases para el tratamiento de información Parte 1.

5. Clasificación de los Streams: Tratamiento de información

Utilidad

Datos primitivos de Java

Análisis

DataInputStream,
DataOutputStream

Permite leer/escribir datos primitivos de Java como
int, float, long y double

5. Clasificación de los Streams: Tratamiento de información

```
try {
    DataOutputStream dataOutputStream = new DataOutputStream(
        new FileOutputStream("tema2_1.txt"));

    dataOutputStream.write(65);
    dataOutputStream.writeInt(12345);
    dataOutputStream.writeFloat(3.14f);
    dataOutputStream.writeDouble(2.718);
    dataOutputStream.close();

    DataInputStream dataInputStream = new DataInputStream(
        new FileInputStream("tema2_1.txt"));
    int aByte = dataInputStream.read();
    System.out.println(aByte);

    int anInt = dataInputStream.readInt();
    System.out.println(anInt);

    float aFloat = dataInputStream.readFloat();
    System.out.println(aFloat);

    double aDouble = dataInputStream.readDouble();
    System.out.println(aDouble);

    dataInputStream.close();
} catch (FileNotFoundException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
}
```

Agradecimientos

Material adaptado a partir del trabajo de:
Antonio Cuadros Lapresa

*“Cuando el conocimiento se comparte, se multiplica” -
Thomas Thredgold*